THỰC HÀNH 2: TƯ DUY HƯỚNG ĐỐI TƯỢNG (OOP CORE)

🎯 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Thiết kế Class** với attributes và methods đúng cách
2. ✅ **Hiểu Encapsulation** và tại sao phải dùng `private`
3. ✅ **Sử dụng Constructors** (default, overloaded, chaining)
4. ✅ **Viết Getters/Setters** với validation
5. ✅ **Phân biệt** Instance vs Static members
6. ✅ **Áp dụng** từ khóa `this` trong thực tế

⏱️ **Thời gian ước tính:** 5-7 giờ

📋 **Kiến thức cần có trước:**

- Đã hoàn thành Thực hành 1
- Đã đọc Chương 2: Class & Object
- Hiểu cơ bản về method, variable scope

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 2.1: Quản Lý Tài Khoản Ngân Hàng (BankAccount)

Mục tiêu: Xây dựng class chuẩn chỉnh với Encapsulation

Bước 1: Phân Tích Yêu Cầu

Tài khoản ngân hàng cần có:

- Số tài khoản (accountNumber)
- Tên chủ tài khoản (ownerName)
- Số dư (balance)

Các hành động:

- Nạp tiền (deposit)
- Rút tiền (withdraw)
- Xem số dư (getBalance)
- Hiển thị thông tin (display)

Bước 2: Thiết Kế Class

Tạo file: `BankAccount.java` trong package `com.bootcamp.lab2`

```java
package com.bootcamp.lab2;

public class BankAccount {
    // ATTRIBUTES (Thuộc tính) - Luôn để PRIVATE
    private String accountNumber;
    private String ownerName;
    private double balance;
```

Giải thích:

- `private`: Che giấu dữ liệu, không cho truy cập trực tiếp từ bên ngoài
- Đây là nguyên lý **Encapsulation** (Đóng gói)

Bước 3: Viết Constructor

```java
    // CONSTRUCTOR - Hàm khởi tạo
    public BankAccount(String accNum, String name, double initialBalance)
{
        this.accountNumber = accNum;
        this.ownerName = name;

        // Validation: Số dư ban đầu không được âm
        if (initialBalance < 0) {
            System.out.println("⚠ Số dư ban đầu không được âm. Đã set về
0.");
            this.balance = 0;
        } else {
            this.balance = initialBalance;
        }
    }
```

Giải thích:

1. **Constructor là gì?**
   - Method đặc biệt, tên trùng với tên class
   - Tự động gọi khi tạo object: `new BankAccount(...)`
   - Dùng để khởi tạo giá trị ban đầu
2. **Từ khóa `this`:**
   - `this.accountNumber`: Thuộc tính của object
   - `accNum`: Tham số truyền vào
   - Phân biệt giữa thuộc tính và tham số cùng tên
3. **Validation trong Constructor:**
   - Kiểm tra dữ liệu ngay từ đầu
   - Đảm bảo object luôn ở trạng thái hợp lệ

Bước 4: Viết Method Nạp Tiền

```
    // METHOD: Nạp tiền
    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("❌ Số tiền nạp phải lớn hơn 0!");
            return;  // Dừng hàm ngay lập tức
        }

        this.balance += amount;
        System.out.println("✅ Đã nạp " + amount + " VND vào TK " +
accountNumber);
        System.out.println("   Số dư hiện tại: " + this.balance + " VND");
    }
```

Giải thích:

- `void`: Không trả về giá trị
- `return;`: Thoát hàm ngay (không cần giá trị vì là void)
- Validation: Chỉ cho phép nạp số dương

Bước 5: Viết Method Rút Tiền

```
    // METHOD: Rút tiền
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("❌ Số tiền rút phải lớn hơn 0!");
            return;
        }

        if (amount > this.balance) {
            System.out.println("❌ Số dư không đủ!");
            System.out.println("   Số dư hiện có: " + this.balance + "
VND");
            System.out.println("   Số tiền muốn rút: " + amount + " VND");
            return;
        }

        this.balance -= amount;
        System.out.println("✅ Đã rút " + amount + " VND");
        System.out.println("   Số dư còn lại: " + this.balance + " VND");
    }
```

Giải thích:

- Kiểm tra 2 điều kiện:
    1. Số tiền rút > 0
    2. Số tiền rút <= số dư

- Chỉ trừ tiền khi cả 2 điều kiện đều đúng

Bước 6: Viết Getter

```
    // GETTER: Cho phép xem số dư, nhưng KHÔNG cho sửa trực tiếp
    public double getBalance() {
        return this.balance;
    }

    public String getAccountNumber() {
        return this.accountNumber;
    }

    public String getOwnerName() {
        return this.ownerName;
    }
```

Giải thích:

- Getter: Method để đọc giá trị
- Không có Setter cho `balance` → Bảo vệ dữ liệu
- Chỉ có thể thay đổi `balance` qua `deposit()` và `withdraw()`

Bước 7: Viết Method Hiển Thị

```
    // METHOD: Hiển thị thông tin
    public void display() {
        System.out.println("\n=== THÔNG TIN TÀI KHOẢN ===");
        System.out.println("Số TK: " + accountNumber);
        System.out.println("Chủ TK: " + ownerName);
        System.out.printf("Số dư: %.2f VND\n", balance);
        System.out.println("============================\n");
    }
```

Code Hoàn Chỉnh

```
package com.bootcamp.lab2;

public class BankAccount {
    // ATTRIBUTES - PRIVATE
    private String accountNumber;
    private String ownerName;
    private double balance;

    // CONSTRUCTOR
    public BankAccount(String accNum, String name, double initialBalance)
{
```

```
        this.accountNumber = accNum;
        this.ownerName = name;

        if (initialBalance < 0) {
            System.out.println("⚠ Số dư ban đầu không được âm. Đã set về
0.");
            this.balance = 0;
        } else {
            this.balance = initialBalance;
        }
    }

    // METHOD: Nạp tiền
    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("❌ Số tiền nạp phải lớn hơn 0!");
            return;
        }
        this.balance += amount;
        System.out.println("✅ Đã nạp " + amount + " VND vào TK " +
accountNumber);
    }

    // METHOD: Rút tiền
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("❌ Số tiền rút phải lớn hơn 0!");
            return;
        }
        if (amount > this.balance) {
            System.out.println("❌ Số dư không đủ!");
            return;
        }
        this.balance -= amount;
        System.out.println("✅ Đã rút " + amount + " VND");
    }

    // GETTERS
    public double getBalance() {
        return this.balance;
    }

    public String getAccountNumber() {
        return this.accountNumber;
    }

    public String getOwnerName() {
        return this.ownerName;
    }

    // METHOD: Hiển thị
    public void display() {
        System.out.println("\n=== THÔNG TIN TÀI KHOẢN ===");
        System.out.println("Số TK: " + accountNumber);
```

```
        System.out.println("Chủ TK: " + ownerName);
        System.out.printf("Số dư: %.2f VND\n", balance);
        System.out.println("==========================\n");
    }
}
```

Bước 8: Tạo Class Test

Tạo file: `BankApp.java`

```
package com.bootcamp.lab2;

public class BankApp {
    public static void main(String[] args) {
        System.out.println("=== CHƯƠNG TRÌNH QUẢN LÝ NGÂN HÀNG ===\n");

        // Tạo 2 tài khoản (2 Objects riêng biệt)
        BankAccount acc1 = new BankAccount("TK001", "Nguyễn Văn A",
1000000);
        BankAccount acc2 = new BankAccount("TK002", "Trần Thị B", 500000);

        // Giao dịch với acc1
        System.out.println("--- Giao dịch TK001 ---");
        acc1.deposit(500000);
        acc1.withdraw(200000);
        acc1.withdraw(2000000);  // Thử rút quá số dư
        acc1.display();

        // Giao dịch với acc2
        System.out.println("--- Giao dịch TK002 ---");
        acc2.deposit(300000);
        acc2.display();

        // Mỗi tài khoản có số dư riêng
        System.out.println("Số dư TK001: " + acc1.getBalance() + " VND");
        System.out.println("Số dư TK002: " + acc2.getBalance() + " VND");
    }
}
```

Giải thích:

- `new BankAccount(...)`: Tạo object mới trong bộ nhớ Heap
- `acc1` và `acc2` là 2 object độc lập, không ảnh hưởng nhau
- Mỗi object có bản copy riêng của các thuộc tính

LAB 2.2: Hình Chữ Nhật (Rectangle)

Mục tiêu: Luyện tập Constructor Overloading và Method

Yêu Cầu

Tạo class `Rectangle` với:

- Attributes: `width`, `height` (private, double)
- Constructors:
  - Constructor đầy đủ: `Rectangle(double w, double h)`
  - Constructor mặc định: `Rectangle()` → width = 1, height = 1
- Methods:
  - `getArea()`: Tính diện tích
  - `getPerimeter()`: Tính chu vi
  - `displayInfo()`: Hiển thị thông tin

Hướng Dẫn

Bước 1: Tạo Class và Attributes

```
package com.bootcamp.lab2;

public class Rectangle {
    private double width;
    private double height;
```

Bước 2: Constructor Đầy Đủ

```
    // Constructor đầy đủ
    public Rectangle(double width, double height) {
        // Validation: Chiều dài và rộng phải > 0
        if (width <= 0 || height <= 0) {
            System.out.println("⚠ Chiều dài và rộng phải > 0. Đã set về
1.");
            this.width = 1;
            this.height = 1;
        } else {
            this.width = width;
            this.height = height;
        }
    }
```

Bước 3: Constructor Mặc Định (Overloading)

```
    // Constructor mặc định - Gọi constructor đầy đủ
    public Rectangle() {
        this(1, 1);  // Gọi constructor đầy đủ với giá trị mặc định
    }
```

Giải thích:

- `this(1, 1)`: Gọi constructor khác trong cùng class
- Đây là **Constructor Chaining**
- Tránh lặp code (DRY - Don't Repeat Yourself)

Bước 4: Methods Tính Toán

```
    // Method: Tính diện tích
    public double getArea() {
        return width * height;
    }

    // Method: Tính chu vi
    public double getPerimeter() {
        return 2 * (width + height);
    }
```

Bước 5: Method Hiển Thị

```
    // Method: Hiển thị thông tin
    public void displayInfo() {
        System.out.println("=== HÌNH CHỮ NHẬT ===");
        System.out.printf("Chiều rộng: %.2f\n", width);
        System.out.printf("Chiều dài: %.2f\n", height);
        System.out.printf("Diện tích: %.2f\n", getArea());
        System.out.printf("Chu vi: %.2f\n", getPerimeter());
        System.out.println("=====================\n");
    }

    // GETTERS
    public double getWidth() {
        return width;
    }

    public double getHeight() {
        return height;
    }
}
```

Code Hoàn Chỉnh

```
package com.bootcamp.lab2;

public class Rectangle {
    private double width;
    private double height;
```

```java
    // Constructor đầy đủ
    public Rectangle(double width, double height) {
        if (width <= 0 || height <= 0) {
            System.out.println("⚠ Chiều dài và rộng phải > 0. Đã set về
1.");
            this.width = 1;
            this.height = 1;
        } else {
            this.width = width;
            this.height = height;
        }
    }

    // Constructor mặc định
    public Rectangle() {
        this(1, 1);
    }

    public double getArea() {
        return width * height;
    }

    public double getPerimeter() {
        return 2 * (width + height);
    }

    public void displayInfo() {
        System.out.println("=== HÌNH CHỮ NHẬT ===");
        System.out.printf("Chiều rộng: %.2f\n", width);
        System.out.printf("Chiều dài: %.2f\n", height);
        System.out.printf("Diện tích: %.2f\n", getArea());
        System.out.printf("Chu vi: %.2f\n", getPerimeter());
        System.out.println("=====================\n");
    }

    public double getWidth() {
        return width;
    }

    public double getHeight() {
        return height;
    }
}
```

Test:

```java
package com.bootcamp.lab2;

public class RectangleTest {
    public static void main(String[] args) {
        // Dùng constructor đầy đủ
        Rectangle rect1 = new Rectangle(5.0, 3.0);
```

```
        rect1.displayInfo();

        // Dùng constructor mặc định
        Rectangle rect2 = new Rectangle();
        rect2.displayInfo();

        // Test validation
        Rectangle rect3 = new Rectangle(-5, 3);  // Sẽ set về 1, 1
        rect3.displayInfo();
    }
}
```

LAB 2.3: Sinh Viên với Static Counter

Mục tiêu: Hiểu Instance vs Static members

Yêu Cầu

Tạo class `Student` với:

- Attributes: `name`, `age`, `gpa` (instance variables)
- Static variable: `count` (đếm tổng số sinh viên đã tạo)
- Static constant: `SCHOOL_NAME` = "Đại Học ABC"
- Constructor: Tăng `count` mỗi khi tạo object mới
- Static method: `getTotalStudents()` trả về số lượng sinh viên

Hướng Dẫn

```
package com.bootcamp.lab2;

public class Student {
    // INSTANCE VARIABLES - Mỗi object có bản copy riêng
    private String name;
    private int age;
    private double gpa;

    // STATIC VARIABLE - Dùng chung cho tất cả objects
    private static int count = 0;

    // STATIC CONSTANT - Hằng số dùng chung
    public static final String SCHOOL_NAME = "Đại Học ABC";

    // CONSTRUCTOR
    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;

        // Validation GPA
        if (gpa < 0 || gpa > 10) {
```

```
            System.out.println("⚠ GPA phải từ 0-10. Đã set về 0.");
            this.gpa = 0;
        } else {
            this.gpa = gpa;
        }

        // Tăng biến đếm chung
        count++;
        System.out.println("✅ Đã tạo sinh viên mới. Tổng số: " + count);
    }

    // STATIC METHOD - Truy cập qua tên class
    public static int getTotalStudents() {
        return count;
    }

    // INSTANCE METHODS
    public void displayInfo() {
        System.out.println("=== THÔNG TIN SINH VIÊN ===");
        System.out.println("Tên: " + name);
        System.out.println("Tuổi: " + age);
        System.out.println("GPA: " + gpa);
        System.out.println("Trường: " + SCHOOL_NAME);
        System.out.println("============================\n");
    }

    // GETTERS
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getGpa() { return gpa; }
}
```

Test:

```
package com.bootcamp.lab2;

public class StudentTest {
    public static void main(String[] args) {
        // Truy cập static constant
        System.out.println("Trường: " + Student.SCHOOL_NAME);
        System.out.println("Số SV hiện tại: " + Student.getTotalStudents()
+ "\n");

        // Tạo các sinh viên
        Student sv1 = new Student("Nguyễn Văn A", 20, 8.5);
        Student sv2 = new Student("Trần Thị B", 21, 7.8);
        Student sv3 = new Student("Lê Văn C", 19, 9.2);

        // Tất cả đều dùng chung biến count
        System.out.println("\nTổng số sinh viên: " +
Student.getTotalStudents());
```

```
        // Mỗi sinh viên có thông tin riêng
        sv1.displayInfo();
        sv2.displayInfo();
        sv3.displayInfo();
    }
}
```

Giải thích:

1. **Instance Variables (`name`, `age`, `gpa`):**
   - Mỗi object có bản copy riêng
   - `sv1.name` khác `sv2.name`
2. **Static Variable (`count`):**
   - Chỉ có 1 bản copy duy nhất
   - Tất cả objects dùng chung
   - Tăng mỗi khi tạo object mới
3. **Static Method (`getTotalStudents()`):**
   - Truy cập qua tên class: `Student.getTotalStudents()`
   - Không cần tạo object

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 2.1: Class Sinh Viên với Validation

Đề bài: Tạo class `Student` với:

- Attributes: `name`, `mark` (điểm 0-10)
- Method `setMark(double m)`:
  - Nếu `m < 0` hoặc `m > 10` → In lỗi, không gán
  - Nếu hợp lệ → Gán giá trị
- Method `getRank()`:
  - Trả về "Xuất sắc" (>=9), "Giỏi" (>=8), "Khá" (>=6.5), "Trung bình" (còn lại)

Hướng Dẫn

```
package com.bootcamp.lab2;

public class Student {
    private String name;
    private double mark;

    public Student(String name, double mark) {
        this.name = name;
        setMark(mark);  // Dùng setter để validation
```

```
    }

    // SETTER với validation
    public void setMark(double m) {
        if (m < 0 || m > 10) {
            System.out.println("❌ Điểm phải từ 0 đến 10. Không thể gán
giá trị " + m);
            // Không gán giá trị
        } else {
            this.mark = m;
            System.out.println("✅ Đã cập nhật điểm cho " + name + ": " +
m);
        }
    }

    // GETTER
    public double getMark() {
        return mark;
    }

    public String getName() {
        return name;
    }

    // METHOD: Xếp loại
    public String getRank() {
        if (mark >= 9) {
            return "Xuất sắc";
        } else if (mark >= 8) {
            return "Giỏi";
        } else if (mark >= 6.5) {
            return "Khá";
        } else {
            return "Trung bình";
        }
    }

    public void displayInfo() {
        System.out.println("=== THÔNG TIN SINH VIÊN ===");
        System.out.println("Tên: " + name);
        System.out.println("Điểm: " + mark);
        System.out.println("Xếp loại: " + getRank());
        System.out.println("==========================\n");
    }
}
```

Test:

```
package com.bootcamp.lab2;

public class StudentTest {
    public static void main(String[] args) {
```

```
        Student sv1 = new Student("Nguyễn Văn A", 9.5);
        sv1.displayInfo();

        Student sv2 = new Student("Trần Thị B", 7.8);
        sv2.displayInfo();

        // Test validation
        sv2.setMark(15);  // ❌ Lỗi
        sv2.setMark(8.5);  // ✅ OK
        sv2.displayInfo();
    }
}
```

Bài 2.2: Class Circle

Đề bài: Tạo class `Circle` với:

- Attribute: `radius` (private, double)
- Constructor: Nhận radius, nếu < 0 thì set = 1
- Methods:
  - `getArea()`: Diện tích = π × r²
  - `getCircumference()`: Chu vi = 2 × π × r
  - `displayInfo()`: Hiển thị thông tin

Hướng Dẫn

```
package com.bootcamp.lab2;

public class Circle {
    private double radius;

    // Hằng số PI
    private static final double PI = 3.14159;

    public Circle(double radius) {
        if (radius < 0) {
            System.out.println("⚠ Bán kính không được âm. Đã set về 1.");
            this.radius = 1;
        } else {
            this.radius = radius;
        }
    }

    public double getArea() {
        return PI * radius * radius;
    }

    public double getCircumference() {
        return 2 * PI * radius;
```

```java
    }

    public void displayInfo() {
        System.out.println("=== HÌNH TRÒN ===");
        System.out.printf("Bán kính: %.2f\n", radius);
        System.out.printf("Diện tích: %.2f\n", getArea());
        System.out.printf("Chu vi: %.2f\n", getCircumference());
        System.out.println("================\n");
    }

    public double getRadius() {
        return radius;
    }
}
```

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 3.1: Class Product (Sản Phẩm)

Đề bài: Tạo class `Product` với:

- Attributes: `id`, `name`, `price`, `quantity` (tất cả private)
- Constructor đầy đủ
- Getters và Setters với validation:
  - `price >= 0`
  - `quantity >= 0`
  - `name` không được rỗng
- Method `getTotalValue()`: Tính tổng giá trị = price × quantity
- Method `displayInfo()`: Hiển thị thông tin

Yêu cầu:

- Validation đầy đủ
- Thông báo lỗi rõ ràng

Bài 3.2: Class Time (Thời Gian)

Đề bài: Tạo class `Time` biểu diễn thời gian (giờ:phút:giây):

- Attributes: `hour`, `minute`, `second` (private, int)
- Constructor: Nhận hour, minute, second
- Validation:
  - `hour`: 0-23
  - `minute`: 0-59
  - `second`: 0-59
- Methods:
  - `getTotalSeconds()`: Tính tổng số giây
  - `displayTime()`: Hiển thị dạng HH:MM:SS

    - `addSeconds(int s)`: Thêm s giây vào thời gian hiện tại

Ví dụ:

- Input: `Time(23, 59, 50)`
- `addSeconds(15)` → `00:00:05` (tràn sang ngày mới)

Bài 3.3: Class Book (Sách)

Đề bài: Tạo class `Book` với:

- Attributes: `isbn`, `title`, `author`, `price`, `year` (private)
- Constructor đầy đủ
- Static variable: `totalBooks` (đếm tổng số sách)
- Methods:
    - `isOldBook()`: Trả về true nếu sách xuất bản trước 2000
    - `getDiscountPrice(double discountPercent)`: Tính giá sau giảm
    - `displayInfo()`: Hiển thị thông tin

Yêu cầu:

- Tăng `totalBooks` mỗi khi tạo sách mới
- Static method `getTotalBooks()` trả về tổng số sách

Bài 3.4: Class Point (Điểm)

Đề bài: Tạo class `Point` biểu diễn điểm trong mặt phẳng:

- Attributes: `x`, `y` (private, double)
- Constructor: Nhận x, y
- Methods:
    - `distanceToOrigin()`: Khoảng cách đến gốc tọa độ
    - `distanceTo(Point other)`: Khoảng cách đến điểm khác
    - `displayPoint()`: Hiển thị dạng (x, y)

Công thức:

- Khoảng cách đến gốc: $\sqrt{(x^2 + y^2)}$
- Khoảng cách giữa 2 điểm: $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$

Bài 3.5: Class Fraction (Phân Số)

Đề bài: Tạo class `Fraction` biểu diễn phân số:

- Attributes: `numerator` (tử số), `denominator` (mẫu số) - private, int
- Constructor: Nhận tử số và mẫu số
- Validation: Mẫu số không được = 0
- Methods:
    - `add(Fraction other)`: Cộng 2 phân số, trả về Fraction mới

    - `multiply(Fraction other)`: Nhân 2 phân số, trả về Fraction mới
    - `simplify()`: Rút gọn phân số (tìm UCLN)
    - `display()`: Hiển thị dạng "tử/mẫu"

Gợi ý:

- Cộng: (a/b) + (c/d) = (ad + bc) / bd
- Nhân: (a/b) × (c/d) = (ac) / (bd)
- UCLN: Dùng thuật toán Euclid

Bài 3.6: Class Counter với Static

Đề bài: Tạo class `Counter` đếm số lượng instance đã tạo:

- Static variable: `totalCount` (tổng số Counter đã tạo)
- Instance variable: `id` (ID duy nhất của mỗi Counter)
- Constructor: Tự động gán ID và tăng `totalCount`
- Static method: `getTotalCount()` trả về tổng số
- Method: `getId()` trả về ID của instance này

Gợi ý:

- Mỗi lần tạo Counter mới, tăng `totalCount` và gán ID = `totalCount`
- Static variable được chia sẻ bởi tất cả instances

Yêu cầu:

- Tạo 5 Counter, kiểm tra ID và totalCount

Bài 3.7: Class Temperature Converter

Đề bài: Tạo class `Temperature` với:

- Attribute: `celsius` (private, double) - nhiệt độ theo độ C
- Constructor: Nhận giá trị độ C
- Methods:
    - `toFahrenheit()`: Chuyển sang độ F (F = C × 9/5 + 32)
    - `toKelvin()`: Chuyển sang độ K (K = C + 273.15)
    - `setCelsius(double c)`: Set nhiệt độ mới với validation (>= -273.15)
    - `displayAll()`: Hiển thị cả 3 đơn vị

Gợi ý:

- -273.15°C là độ không tuyệt đối (0K)
- Validation: Không cho phép nhiệt độ < -273.15°C

Bài 3.8: Class Rectangle với Methods Nâng Cao

Đề bài: Mở rộng class `Rectangle` với các methods:

- `isSquare()`: Kiểm tra có phải hình vuông không
- `scale(double factor)`: Phóng to/thu nhỏ (nhân width và height với factor)
- `rotate()`: Xoay 90 độ (hoán đổi width và height)
- `canFitInside(Rectangle other)`: Kiểm tra rectangle này có thể đặt vừa trong rectangle khác không

Gợi ý:

- `isSquare()`: width == height
- `scale()`: Nhân cả width và height với factor (factor > 0)
- `rotate()`: Hoán đổi width và height
- `canFitInside()`: width <= other.width && height <= other.height

Bài 3.9: Class Student với Static Methods

Đề bài: Tạo class `Student` với:

- Attributes: `name`, `mathScore`, `physicsScore`, `chemistryScore` (private)
- Constructor đầy đủ
- Instance methods:
  - `getAverage()`: Tính điểm trung bình 3 môn
  - `getGrade()`: Xếp loại (A: >=8, B: >=6.5, C: >=5, D: <5)
- Static methods:
  - `getClassAverage(Student[] students)`: Tính điểm trung bình của cả lớp
  - `getTopStudent(Student[] students)`: Tìm học sinh có điểm trung bình cao nhất

Gợi ý:

- Static methods nhận mảng Student làm tham số
- Duyệt mảng để tính toán
- Validation: Điểm từ 0-10

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: Quản Lý Thư Viện (Array of Objects)

Đề bài: Viết chương trình quản lý Sách trong thư viện.

Yêu cầu:

1. **Class `Book`:**
   - Attributes: `title`, `author`, `isBorrowed` (boolean)
   - Constructor, Getters, Setters
   - Method `borrow()`: Cho mượn (set isBorrowed = true)
   - Method `returnBook()`: Trả sách (set isBorrowed = false)
2. **Class `Library`:**

    - Attribute: `books[]` (mảng chứa Book)
    - Constructor: Khởi tạo mảng với 5 cuốn sách
    - Method `findBook(String title)`: Tìm sách theo tên
    - Method `borrowBook(String title)`:
        - Tìm sách
        - Nếu tìm thấy và chưa mượn → Cho mượn
        - Nếu đã mượn → Báo "Hết hàng"
        - Nếu không tìm thấy → Báo "Không có sách này"

3. **Class `LibraryApp`:**

    - Menu:

```
1. Tìm sách
2. Mượn sách
3. Trả sách
4. Hiển thị tất cả sách
5. Thoát
```

Gợi ý:

- Dùng vòng lặp `for` để duyệt mảng
- Dùng `equals()` để so sánh chuỗi (không dùng `==`)

Challenge 2: Constructor Chaining Nâng Cao

Đề bài: Tạo class `Student` với nhiều constructor:

1. `Student(String name, int age, String email)` - Constructor đầy đủ
2. `Student(String name, String email)` - Mặc định age = 18
3. `Student(String name)` - Mặc định age = 18, email = "no-email@school.edu"

Yêu cầu:

- Sử dụng constructor chaining (`this()`)
- Validation: age >= 0 và <= 100
- Validation: email phải chứa "@"

Mục tiêu: Tránh lặp code (DRY)

Challenge 3: Class Tam Giác Hợp Lệ

Đề bài: Tạo class `Triangle` với:

- Attributes: `a`, `b`, `c` (3 cạnh, private, double)
- Constructor: Nhận 3 cạnh
- Validation: Kiểm tra tính hợp lệ của tam giác
    - Tổng 2 cạnh bất kỳ phải > cạnh còn lại

    - `a + b > c && b + c > a && a + c > b`
    - Nếu không hợp lệ → In lỗi và set tất cả = 0
- Methods:
    - `getType()`: Trả về loại tam giác
        - "Tam giác đều" (a == b == c)
        - "Tam giác cân" (2 cạnh bằng nhau)
        - "Tam giác vuông" ($a^2 + b^2 == c^2$ hoặc tương tự)
        - "Tam giác thường" (còn lại)
    - `getArea()`: Tính diện tích (dùng công thức Heron)
    - `getPerimeter()`: Tính chu vi

Công thức Heron:

```
s = (a + b + c) / 2
area = √(s × (s−a) × (s−b) × (s−c))
```

📚 BẢNG GHI NHỚ (CHEAT SHEET)

Access Modifiers

Modifier	Truy cập từ	Khi nào dùng
`private`	Chỉ trong class	**Hầu hết attributes**
`public`	Mọi nơi	Methods cần cho bên ngoài, main
`protected`	Class + Subclass	Sẽ học ở Chương 3
(không có)	Trong package	Ít dùng

Constructor

Loại	Cú pháp	Ví dụ
Default	`public ClassName() { }`	Tự động nếu không có constructor nào
Parameterized	`public ClassName(params) { }`	Nhận tham số
Chaining	`this(...)`	Gọi constructor khác

Getter/Setter Pattern

```java
// Getter
public returnType getFieldName() {
    return fieldName;
}

// Setter với validation
```

```
public void setFieldName(type value) {
    if (validation) {
        this.fieldName = value;
    } else {
        // Báo lỗi
    }
}
```

Static vs Instance

Đặc điểm	Instance	Static
Số lượng	Mỗi object 1 copy	Chỉ 1 copy cho cả class
Truy cập	`object.field`	`ClassName.field`
Khi nào dùng	Dữ liệu riêng của object	Dữ liệu dùng chung

✅ CHECKLIST HOÀN THÀNH

- ☐ Đã hiểu tại sao phải dùng `private` cho attributes
- ☐ Đã viết được constructor với validation
- ☐ Đã sử dụng constructor chaining
- ☐ Đã viết getters/setters đúng cách
- ☐ Đã phân biệt được instance vs static
- ☐ Đã hoàn thành LAB 2.1 (BankAccount)
- ☐ Đã hoàn thành LAB 2.2 (Rectangle)
- ☐ Đã hoàn thành LAB 2.3 (Student với Static)
- ☐ Đã làm ít nhất 2 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Thiết kế Class** với Encapsulation đúng cách
2. ✅ **Sử dụng Constructors** (default, overloaded, chaining)
3. ✅ **Viết Getters/Setters** với validation
4. ✅ **Phân biệt** Instance vs Static members
5. ✅ **Áp dụng** từ khóa `this` trong thực tế
6. ✅ **Tư duy OOP** cơ bản

Bước tiếp theo: Chuyển sang Thực hành 3: Kế thừa & Đa hình

Chúc bạn học tốt! 🚀